Module 1 SE QB solved

1. Define Software. List and Explain the Categories of Software.

Definition of Software

Software is a collection of programs, data structures, and descriptive information that enables a computer system to perform the desired functions and operations.

The three essential elements of software are:

- Programs: Instructions that provide desired features, functions, and performance.
- Data structures: Structures that enable programs to manipulate information.
- Descriptive information: Hard-copy or virtual information describing the operation and use of software.

Software is a logical rather than a physical system element.

Categories of Software

According to the notes, software is classified into the following seven categories:

1. System Software

System software serves other programs and provides the basic facilities required for computer operation. It includes operating system components, device drivers, and utilities.

2. Application Software

Application software is developed to solve a specific business or technical need. It helps users perform particular tasks.

3. Engineering and Scientific Software

This software uses computational algorithms to solve scientific and engineering problems. It supports complex calculations, simulations, and technical analysis.

4. Embedded Software

Embedded software resides within a product or system and controls its features and functions. It is commonly used in electronic devices and control systems.

5. Product-Line Software

Product-line software provides a particular capability to many different customers. It is designed for use by multiple users or organizations with similar requirements.

6. Web Applications (WebApps)

Web applications are network-centric applications delivered through web technologies and browsers. They range from simple web pages to sophisticated systems and combine software functionality with content and presentation.

7. Artificial Intelligence Software

Artificial intelligence software uses nonnumerical algorithms to solve complex problems. It enables systems to perform intelligent tasks such as reasoning, learning, and decision-making.

2. Explain the Unique Nature of WebApps with Their Attributes.

Definition

Web applications (WebApps) are network-centric software applications delivered through web technologies and browsers. They combine software functionality with content and presentation.

According to Pressman, WebApps exhibit a unique combination of characteristics that distinguish them from conventional software applications.

Attributes of WebApps

1. Network Intensiveness

WebApps operate over a network, such as the Internet or an intranet. They depend on network connectivity to communicate with users and exchange information.

2. Concurrency

Many users may access a WebApp simultaneously. The application must handle multiple users and their requests efficiently.

3. Unpredictable Load

The number of users accessing a WebApp can vary greatly. The system must handle sudden increases or decreases in traffic.

4. Performance

Users expect WebApps to respond quickly. Slow response times can reduce user satisfaction and affect the application's effectiveness.

5. Availability

WebApps are often expected to be available 24 hours a day, 7 days a week, and 365 days a year (24/7/365).

6. Data Driven

WebApps frequently provide access to databases and dynamically generated information. Their functionality depends on storing, retrieving, processing, and presenting data.

7. Content Sensitive

The quality, accuracy, and relevance of information presented by a WebApp are important. Content plays a major role in determining the application's overall quality.

8. Continuous Evolution

WebApps are frequently modified to meet changing user requirements, improve functionality, update content, and adapt to new technologies.

9. Immediacy

WebApps often need to be developed, deployed, and updated within short time-to-market periods. Rapid delivery is an important characteristic.

10. Security

Since WebApps are accessible through networks, protecting data and infrastructure is essential. Security measures are required to prevent unauthorized access and attacks.

11. Aesthetics

The appearance and presentation of a WebApp are important. An attractive interface, proper layout, and good visual design improve the user experience.

3. Describe the Generic Process Model with Its Five Framework Activities and Umbrella Activities.

Definition

A generic software process model is a framework that organizes the activities, actions, tasks, and work products involved in software development. It consists of five framework activities, umbrella activities, and a suitable process flow.

1. Five Framework Activities

The generic process model consists of the following five framework activities.

1. Communication

The communication activity involves understanding stakeholders and gathering requirements. Developers interact with customers to identify their needs, objectives, and expectations.

2. Planning

Planning defines the tasks, resources, work products, risks, and schedule required for the project. It helps organize and control the software development process.

3. Modeling

Modeling involves building models to understand requirements and design the software. It includes analysis and software design to establish the structure and behavior of the system.

4. Construction

Construction involves generating the source code and performing the testing needed to uncover errors. It ensures that the software is implemented according to the design and requirements.

5. Deployment

Deployment involves delivering the software or its increment to users, obtaining their evaluation, and collecting feedback. The feedback helps improve subsequent versions of the software.

2. Umbrella Activities

Umbrella activities are carried out throughout the software development process. They help manage progress, quality, changes, risks, and work products.

The major umbrella activities are:

1. Software Project Tracking and Control

It monitors project progress and compares actual progress with the planned schedule. Corrective actions are taken whenever necessary.

2. Risk Management

It identifies, analyzes, and manages potential risks that may affect the project's cost, schedule, quality, or successful completion.

3. Software Quality Assurance (SQA)

It ensures that software processes and work products follow the required quality standards.

4. Technical Reviews

Technical reviews examine software work products to identify errors, defects, and inconsistencies at an early stage.

5. Measurement

It collects and analyzes measurements related to software processes and products to evaluate progress, quality, and performance.

6. Software Configuration Management

It manages changes to software components, documents, and other work products. It maintains consistency and controls different versions of the software.

7. Reusability Management

It identifies and manages reusable software components to reduce development effort, time, and cost.

8. Work Product Preparation and Production

It involves preparing and producing the required documents, reports, models, and other deliverables throughout the software process.

3. Diagram of the Generic Process Model

The diagram shows the five framework activities in sequence, along with task sets and umbrella activities.



The task set associated with the process contains work tasks, work products, quality assurance points, and project milestones. Umbrella activities operate across the framework activities.

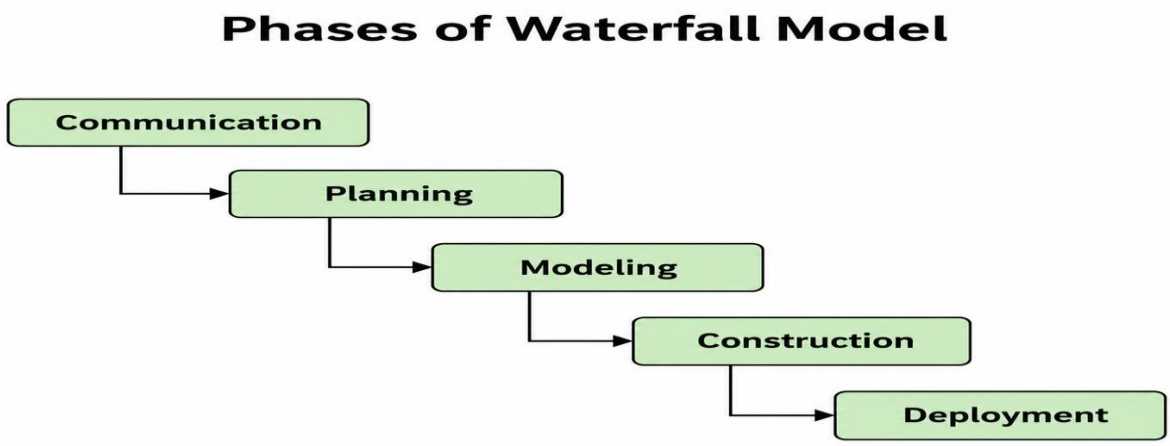
4. Explain the Waterfall Model with a Neat Diagram. Mention Its Advantages and Drawbacks.

Definition

The Waterfall Model is a prescriptive software process model in which software development activities are carried out sequentially, with each phase generally completed before the next phase begins.

It is best suited for projects in which the requirements are well understood and reasonably stable.

Phases of the Waterfall Model



1. Communication

The requirements are collected by communicating with customers and stakeholders. The objectives and expected functionality of the software are identified.

2. Planning

The project schedule, resources, tasks, risks, and estimated costs are determined. A plan is prepared to guide the development process.

3. Modeling

The software requirements are analyzed, and the system design is prepared. The architecture, components, and interactions of the system are defined.

4. Construction

The design is converted into source code. Testing is performed to identify and correct errors in the software.

5. Deployment

The completed software is delivered to the customer. Feedback is obtained, and necessary support or maintenance is provided.

Advantages of the Waterfall Model

- Simple and systematic: The model is easy to understand and follows a clearly defined sequence of activities.
- Easy to manage: Each phase has specific objectives and deliverables, making project management easier.
- Predictable workflow: It provides a predictable development process when requirements are stable.
- Clear documentation: Requirements, designs, and other work products can be documented phase by phase.
- Suitable for stable requirements: It works well when requirements are well understood and unlikely to change significantly.

Drawbacks of the Waterfall Model

- Rigid to changes: It is difficult to accommodate changing requirements once development has progressed.
- Late feedback: Customer feedback and evaluation may arrive late in the development process.
- Late testing: Major defects may be discovered only after substantial development work has been completed.
- Limited flexibility: Returning to earlier phases can be difficult and costly.
- Unsuitable for uncertain requirements: It is less appropriate for projects where requirements are unclear or frequently changing.

5. Compare the Incremental and Evolutionary (Prototyping and Spiral) Process Models.

1. Incremental Process Model

The Incremental Process Model delivers a series of operational releases called increments, where each increment adds new functionality to the existing product. It allows the customer to receive useful software early and evaluate the delivered increments.

2. Evolutionary Process Models

Evolutionary models are iterative and produce increasingly complete versions of software. The two important evolutionary models are:

A. Prototyping Model

The Prototyping Model is useful when requirements are unclear or developers are uncertain about an algorithm, operating environment, or human–machine interaction. A prototype is developed, and stakeholder feedback is used to refine the requirements.

B. Spiral Model

The Spiral Model combines the iterative nature of prototyping with the systematic aspects of the Waterfall Model. It follows a risk-driven evolutionary approach, where each revolution considers risks and produces a more complete release.

3. Comparison of Incremental, Prototyping and Spiral Models

Feature	**Incremental Model**	**Prototyping Model**	**Spiral Model**
Basic concept	Delivers software in a series of increments.	Develops a prototype to understand and refine requirements.	Develops software through repeated cycles with risk analysis.
Requirements	Suitable when the system can be divided into functional increments.	Suitable when requirements are unclear or uncertain.	Suitable for large-scale projects with significant risks.
Development approach	Repeated linear sequences, each delivering additional functionality.	Communication, quick planning, quick design, prototype, deployment and feedback.	Repeated cycles involving communication, planning, modeling/construction and risk analysis.
Customer feedback	Feedback on early increments guides later development.	Feedback helps clarify and refine requirements.	Stakeholder commitment and feedback support successive releases.

Risk management	Helps manage staffing and technical risks through increments.	Risks include mistaking prototype quality for production quality.	Risk analysis is a central activity in every revolution.
Main advantage	Operational capability reaches customers early.	Makes the intended system tangible and helps discover unclear requirements.	Supports systematic development with explicit risk management.
Main drawback	Requires planning and coordination of increments.	Quick compromises may become permanent; the prototype may be mistaken for the final product.	Success depends strongly on effective risk assessment and expertise.

6. Explain the Unified Process Phases and Their Work Products.

Definition

The Unified Process (UP) is an iterative and incremental software development framework. It is use-case-driven, architecture-centric, and adaptable to the needs of a project.

Phases of the Unified Process and Their Work Products

1. Inception Phase

The inception phase establishes the scope and objectives of the project. Stakeholders and business requirements are identified, and the feasibility of the project is considered.

Work products:

- Business requirements
- Preliminary use cases

- Rough software architecture
- Project plan, including resources, risks, and schedule

2. Elaboration Phase

The elaboration phase refines the requirements, architecture, and risks. It establishes an architectural baseline and prepares a more detailed plan for development.

Work products:

- Refined use cases
- Five architectural views
- Executable architectural baseline, when needed
- Reviewed project plan

3. Construction Phase

The construction phase focuses on implementing the software features. The requirements and design models are completed, and the software is developed and tested.

Work products:

- Completed requirements and design models
- Implemented software features
- Unit test results
- Integration test results
- Acceptance test results

4. Transition Phase

The transition phase delivers the software to users. Beta testing is performed, feedback is collected, and the software is prepared for practical use.

Work products:

- Beta testing results
- User feedback
- User manuals and troubleshooting information
- Installation support
- Usable software release

5. Production Phase

The production phase focuses on monitoring the operation of the software and providing ongoing support after release.

Work products:

- Operational monitoring information

- Support infrastructure
- Evaluations of defects
- Change requests

Important Note

The phases of the Unified Process do not have to occur strictly in sequence. Construction, transition, and production can overlap for different increments.

7. Discuss the Personal Software Process (PSP) and Team Software Process (TSP).

1. Personal Software Process (PSP)

The Personal Software Process (PSP) is a disciplined, metrics-based approach that emphasizes personal measurement, planning, and quality control in software development.

It helps individual software engineers improve their performance by collecting and analyzing data about their work.

Activities of PSP

1. Planning: Estimate software size and resources, defects, tasks, and schedule based on requirements.
2. High-Level Design: Prepare external specifications and component design. Develop prototypes when requirements or technical aspects are uncertain.
3. High-Level Design Review: Perform formal verification to identify design errors and maintain metrics.
4. Development: Refine the design, generate, review, compile, and test code while maintaining metrics.
5. Postmortem: Analyze collected measurements and use the results to improve the personal development process.

2. Team Software Process (TSP)

The Team Software Process (TSP) aims to build a self-directed project team that organizes itself to produce high-quality software. It uses defined roles, shared goals, quantitative project data, and systematic project monitoring.

Characteristics of TSP

1. Shared Goals: All team members work toward common project objectives.
2. Defined Roles: Responsibilities are clearly assigned to team members.

3. Quantitative Project Data: Measurements are collected to monitor project progress and quality.
4. Local Standards: Team members follow established standards to maintain consistency.
5. Risk Assessment: Potential project risks are identified and assessed.
6. Project Status Tracking: Progress is monitored to ensure that the project remains on track.

Framework Activities of TSP

1. Project Launch: Establish team goals, roles, responsibilities, and the project plan.
2. High-Level Design: Develop the overall software architecture and design.
3. Implementation: Develop the software according to the agreed design and standards.
4. Integration and Test: Integrate software components and test the complete system.
5. Postmortem: Review project results and measurements to identify improvements for future projects.

Scripts, forms, and standards guide team activities. Measurement and analysis support continuous process improvement.

3. Difference Between PSP and TSP

Personal Software Process (PSP)	**Team Software Process (TSP)**
Focuses on an individual software engineer.	Focuses on a self-directed software development team.
Emphasizes personal planning and measurement.	Emphasizes team planning and coordination.
Improves individual performance and quality control.	Aims to produce high-quality software through teamwork.
Tracks individual work and defect measurements.	Tracks quantitative project data and team progress.
Uses personal development activities and reviews.	Uses project launch, design, implementation, integration and testing, and postmortem activities.

Helps engineers improve their personal process.	Uses measurement and analysis for continuous team improvement.

8. List and Explain Software Myths (Management, Customer, Practitioner) and the Realities Behind Them.

Definition

Software myths are erroneous beliefs about software and the process used to develop it. They may sound reasonable, but they can lead to poor decisions, delays, increased costs, and reduced software quality.

Software myths are classified into three categories: management myths, customer myths, and practitioner myths.

1. Management Myths

Management myths are incorrect beliefs held by software project managers about software development and project management.

Myth 1: Standards and procedures alone give developers everything they need.

Reality: Standards and procedures alone are insufficient. Developers also need appropriate methods, tools, training, and effective project management.

Myth 2: Adding more programmers to a late project will help it catch up.

Reality: Adding people creates communication and training overhead. New members must be planned carefully, as they may initially slow down the project.

Myth 3: Outsource the project and then relax.

Reality: Outsourcing does not remove management responsibility. The project still requires monitoring, coordination, communication, and quality control.

2. Customer Myths

Customer myths are incorrect beliefs held by customers about software requirements and development.

Myth 1: A general statement of objectives is enough to start coding.

Reality: Clear and detailed requirements are necessary. Continuous communication between customers and developers helps clarify expectations before and during development.

Myth 2: Software is flexible, so requirement changes are easy at any time.

Reality: Although software can be modified, changes may affect design, code, testing, cost, and schedule. Changes must be evaluated and managed systematically.

3. Practitioner Myths

Practitioner myths are incorrect beliefs held by software developers and other technical professionals.

Myth 1: Once the program works, the job is done.

Reality: Software development continues beyond coding. Testing, documentation, deployment, maintenance, and future improvements are also necessary.

Myth 2: Quality cannot be assessed until the program runs.

Reality: Quality can be evaluated throughout development using requirements reviews, design reviews, technical reviews, and other verification activities.

Myth 3: Only the working program is a deliverable.

Reality: Models, documents, plans, test results, and other work products are also important deliverables that support maintenance and future development.

Myth 4: Software engineering means unnecessary documentation.

Reality: Software engineering emphasizes quality and systematic development. Appropriate documentation and engineering practices can reduce rework and improve delivery time.

9. What Is Agility? Explain How Agility Relates to the Cost of Change.

Definition of Agility

Agility is the ability of a software team to respond appropriately to change while maintaining forward progress.

Agile development treats change as a normal part of software development and emphasizes communication, rapid delivery, and flexible planning.

Characteristics of Agility

- Embrace change: Change is treated as a normal part of software development.

- Facilitate communication: Teams reduce barriers among developers, managers, and business people.
- Deliver rapidly: Operational software is delivered in short, useful increments.
- Keep plans flexible: Planning recognizes uncertainty and allows adaptation when requirements change.

Agility and the Cost of Change

The cost of change refers to the effort, time, and resources required to modify software when requirements or designs change.

Conventional View

In conventional software development, the cost of change tends to increase nonlinearly as development progresses. A change introduced later may affect requirements, design, code, testing, and completed work, making it more expensive to implement.

How Agile Processes Reduce the Cost of Change

Agile processes aim to flatten the cost-of-change curve through the following practices:

1. Incremental delivery: Software is delivered in small increments, allowing changes to be managed within an increment.
2. Continuous unit testing: Frequent testing helps identify defects early and reduces the effort needed for later corrections.
3. Pair programming: Two developers work together, helping detect errors and improve code quality.
4. Frequent feedback: Regular customer evaluation helps identify changing requirements early and reduces costly rework.

Key Idea

Agile processes do not eliminate the cost of change. They aim to make change more manageable by using short development cycles, continuous testing, collaboration, and frequent feedback.

10. With a Neat Diagram, Explain the Extreme Programming (XP) Values, Process, and Practices.

Definition

Extreme Programming (XP) is an agile software development process that emphasizes communication, simplicity, feedback, courage, and respect. It supports rapid development, continuous customer involvement, and frequent delivery of working software.

1. Values of Extreme Programming (XP)

XP is based on five fundamental values.

1. Communication

Communication between developers, customers, and stakeholders is encouraged to ensure that requirements are understood clearly and problems are resolved quickly.

2. Simplicity

The design and implementation are kept as simple as possible. The team focuses on the current requirements instead of adding unnecessary features.

3. Feedback

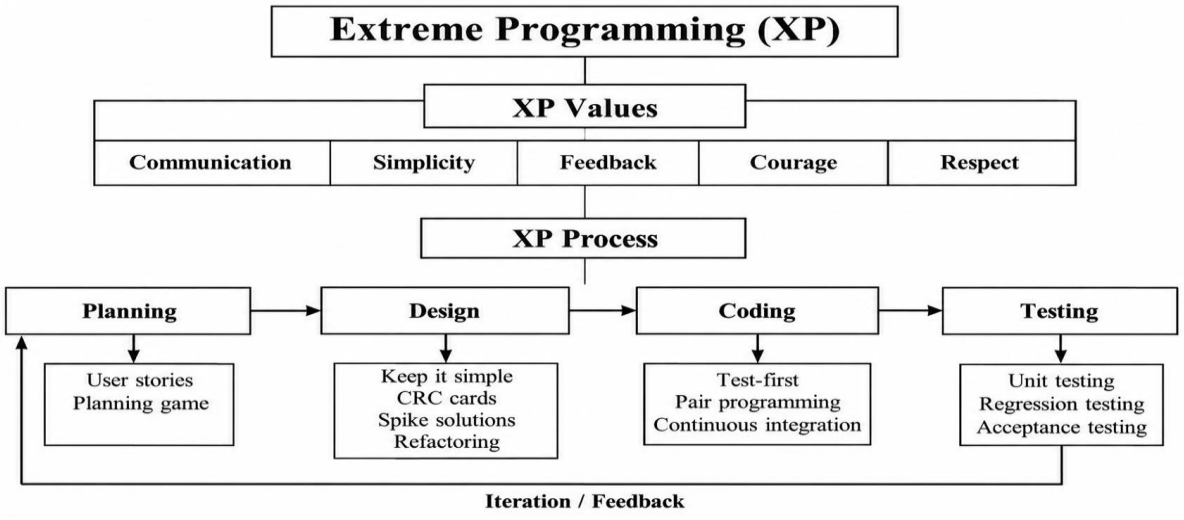
Frequent feedback is obtained through testing, customer evaluation, and regular software releases. This helps the team identify problems and improve the product.

4. Courage

Developers must have the courage to modify designs, remove unnecessary code, accept feedback, and make necessary changes when problems are identified.

5. Respect

Team members respect one another's contributions, communicate openly, and work collaboratively toward common goals.



2. Extreme Programming (XP) Process

XP consists of four framework activities: Planning, Design, Coding, and Testing.

1. Planning

- Listen to stakeholders and understand their requirements.
- Create user stories describing the required features.
- The customer assigns business value or priority to user stories.
- The development team estimates the cost of each story.
- Group stories into releases and use project velocity to plan later releases.

2. Design

- Follow the Keep-It-Simple principle.
- Design only for the requirements of the current user story.
- Use CRC (Class–Responsibility–Collaborator) cards in an object-oriented context.
- Use spike solutions to investigate difficult design problems.
- Refactor the software to improve its internal structure without changing its external behavior.

3. Coding

- Create unit tests before writing the code.
- Use pair programming, in which two developers work at one workstation.
- Perform continuous integration of code changes.
- Encourage real-time problem-solving and code reviews.
- Keep implementation focused on passing the tests.

4. Testing

- Perform automated unit testing.
- Conduct regression testing after changes.
- Perform integration and validation testing frequently.
- Derive acceptance tests from user stories and focus on customer-visible functionality.

Diagram reference: Page 56 of your PDF shows the four XP framework activities. Pages 57–58 provide the detailed planning, design, coding, and testing practices.

3. Practices of Extreme Programming (XP)

The main practices described in your notes are:

1. User Stories: Requirements are expressed as user stories and prioritized by the customer.
2. Planning Game: The customer prioritizes business value, while the team estimates development cost and plans releases.
3. Keep-It-Simple Design: Design is limited to what is required for the current story.
4. Spike Solutions: Small experiments are used to investigate difficult technical or design problems.

5. Refactoring: Internal code structure is improved without changing external behavior.
6. Test-First Development: Unit tests are created before coding.
7. Pair Programming: Two developers work together at one workstation to improve code quality and problem-solving.
8. Continuous Integration: Code changes are integrated frequently to identify integration problems early.
9. Regression Testing: Previously working functionality is tested again after changes.
10. Acceptance Testing: Tests derived from user stories verify customer-visible functionality.

11. Briefly explain any three other Agile Process Models (Scrum, DSDM, Crystal, FDD, Lean Software Development).

1. Scrum

Scrum is an agile process model used to manage software development projects with changing requirements and tight deadlines.

- Product Backlog: A prioritized list of requirements and features that provide business value.
- Sprint: A time-boxed work period used to complete selected backlog requirements.
- Daily Scrum: A short meeting to discuss completed work, obstacles, and upcoming tasks.
- Demo: The working software increment is demonstrated and evaluated by the customer.

Scrum supports frequent feedback, incremental delivery, and adaptation to changing requirements.

2. Dynamic Systems Development Method (DSDM)

DSDM is an agile model that uses time-boxed, incremental prototyping within a controlled project environment.

Its main phases are:

1. Feasibility Study
2. Business Study
3. Functional Model Iteration
4. Design and Build Iteration
5. Implementation

- Functional Model Iteration: Collects user feedback through prototypes.
- Design and Build Iteration: Improves prototypes until they provide operational business value.
- Implementation: Delivers the system; feedback may lead to further iterations.

DSDM emphasizes iterative development, user involvement, and the delivery of business value.

3. Crystal

Crystal is a family of agile process models designed to suit different project characteristics and environments.

- Common Core: Different Crystal methods share basic elements.
- Project-Specific Fit: Roles, practices, patterns, and work products vary according to the project.
- Maneuverability: Focuses on delivering useful working software and preparing for the next development cycle.

Crystal allows teams to select the process that best fits their project size, needs, and working environment.

4. Feature-Driven Development (FDD)

FDD is an agile process model that divides the problem into client-valued features and delivers operational features incrementally. A feature is a client-valued function that can generally be implemented in two weeks or less.

Five activities of FDD:

1. Develop Overall Model: Understand the system and develop an overall model.
2. Build Feature List: Identify and list the required features.
3. Plan by Feature: Prepare a plan for developing each feature.
4. Design by Feature: Design the selected feature.
5. Build by Feature: Implement and test the feature.

5. Lean Software Development (LSD)

Lean Software Development adapts lean-manufacturing principles to software engineering. It aims to reduce waste and improve the speed and quality of software delivery.

Seven principles:

1. Eliminate Waste: Remove unnecessary activities that do not add value.
2. Build Quality In: Maintain quality throughout development.
3. Create Knowledge: Encourage continuous learning.
4. Defer Commitment: Make important decisions when sufficient information is available.
5. Deliver Fast: Deliver useful software quickly.
6. Respect People: Encourage teamwork and trust.
7. Optimize the Whole: Improve the complete development process.

12. Case Study 1: Food-Delivery App

Problem: A food-delivery start-up must launch its mobile app in 8 weeks, and restaurant and customer feedback changes the requirements every week.

Most Suitable Model: Agile/XP

Agile/XP (Extreme Programming) is the most suitable process model because the project has a short deadline and frequently changing requirements. It supports rapid development, continuous feedback, frequent testing, and incremental delivery.

Justification

1. Rapid Development: The team can develop and release essential features quickly to meet the 8-week deadline.
2. Changing Requirements: Agile/XP welcomes changes based on weekly feedback from restaurants and customers.
3. Continuous Feedback: Frequent customer feedback helps the team improve features such as restaurant search, menu browsing, ordering, and payment.
4. Continuous Testing: XP uses automated unit testing, regression testing, and frequent integration to identify defects early.
5. Incremental Delivery: The team can prioritize essential features first and add improvements in subsequent iterations.
6. Customer Satisfaction: Regular feedback ensures that the application meets the actual needs of restaurants and customers.

Comparison with Alternatives

Process model	**Suitability**
Waterfall	Not suitable because requirements change every week. Its sequential phases make changes costly and difficult to accommodate.
Incremental	Suitable because core features can be delivered in increments. However, Agile/XP is preferable when frequent feedback, continuous testing, and rapid adaptation are essential.
Agile/XP	Most suitable because it supports changing requirements, short development cycles, continuous customer feedback, and frequent testing.

Conclusion

Agile/XP is the best choice for this food-delivery start-up because it enables the team to launch within 8 weeks while adapting to weekly feedback. Waterfall is too rigid for frequently changing requirements, whereas Incremental development is a reasonable alternative but Agile/XP provides stronger support for continuous feedback and testing.

13. Case Study 2: Hospital Records System

Problem: A hospital is developing a patient-records system where regulations require complete documentation, and the requirements are fixed and well understood. The team proposes Agile with minimal documentation.

Analysis

Agile with minimal documentation is not the most appropriate choice for this project because hospital records involve sensitive patient information and strict regulatory requirements. Complete documentation is essential for traceability, verification, maintenance, and compliance.

The requirements are already fixed and well understood, so frequent requirement changes and continuous customer-driven modifications are less important.

Recommended Process Model: Waterfall Model

The Waterfall model is a suitable prescriptive process model because it follows a systematic, sequential development approach with clearly defined phases and documentation at each stage.

Phases of the Waterfall Model

1. Communication: Collect and document all hospital requirements, including regulatory and patient-record requirements.
2. Planning: Estimate costs, resources, schedule, and risks.
3. Modeling: Prepare system design, database design, architecture, and other technical documents.
4. Construction: Implement the system and perform unit, integration, and system testing.
5. Deployment: Install the system, provide user training, collect feedback, and perform maintenance.

Justification

- Complete Documentation: Each phase produces documented outputs that support audits and regulatory compliance.
- Stable Requirements: The sequential approach works well because the requirements are fixed and clearly understood.
- Traceability: Requirements can be traced through design, implementation, and testing.
- Quality Assurance: Reviews and testing help identify defects before deployment.
- Maintainability: Detailed design documents and test records support future maintenance and updates.

Why Not Agile with Minimal Documentation?

Agile supports changing requirements and rapid feedback, but a minimal-documentation approach may fail to meet the hospital's documentation and traceability needs. Agile itself is not inherently unsuitable;

it can be adapted to include rigorous documentation. However, given the stable requirements and regulatory demands, Waterfall is the more straightforward choice.

Conclusion

The Waterfall model is recommended because the hospital records system has fixed requirements and demands complete documentation. Its structured phases, documented deliverables, and systematic verification make it more suitable than Agile with minimal documentation.

14. Case Study 3: Net-Banking Project

Problem: A bank's net-banking project is in the testing phase when management requests a new UPI payment feature. The manager also plans to add more programmers to recover lost time.

1. Cost of Change Under Waterfall

In the Waterfall model, development proceeds sequentially through requirements, design, coding, testing, and deployment.

- The project has already reached the testing phase.
- Adding a UPI payment feature requires changes to requirements and possibly the system design, database, and existing code.
- Previously completed work may need to be revised and retested.
- These changes increase development time, cost, and the risk of introducing defects.

The cost-of-change curve shows that the cost of making changes generally increases as the project progresses in a conventional process model.

2. How Agility Reduces the Cost

Agile reduces the cost of change through iterative development, frequent feedback, and continuous testing.

- Incremental Development: Features are developed and delivered in small increments.
- Continuous Testing: Automated unit tests and regression tests help identify problems early.
- Frequent Feedback: Customer feedback is collected throughout development.
- Refactoring: Existing code can be improved to accommodate new requirements.
- Short Iterations: Changes can be prioritized and incorporated into upcoming iterations.

Thus, Agile aims to make the cost-of-change curve flatter compared with conventional development, although changes can still be costly in complex banking systems.

3. Software Myth in the Manager's Plan

The manager believes that adding more programmers to a late project will help recover lost time. This is the management myth of adding people to a delayed software project.

Reality:

- New programmers need time to understand the project and its code.
- Existing programmers must spend time training and coordinating with them.
- Communication and coordination overhead increase.
- In the short term, adding people may delay the project further.

Therefore, adding programmers does not automatically accelerate a delayed project.

Conclusion

Under Waterfall, adding the UPI feature during testing is costly because completed requirements, design, code, and tests may need revision. Agile reduces change costs through short iterations, continuous testing, and feedback. The manager's plan reflects the myth that adding people to a late project will make it finish sooner.